

实验三：实现光线追踪算法与加速算法

2312966-林晖鹏

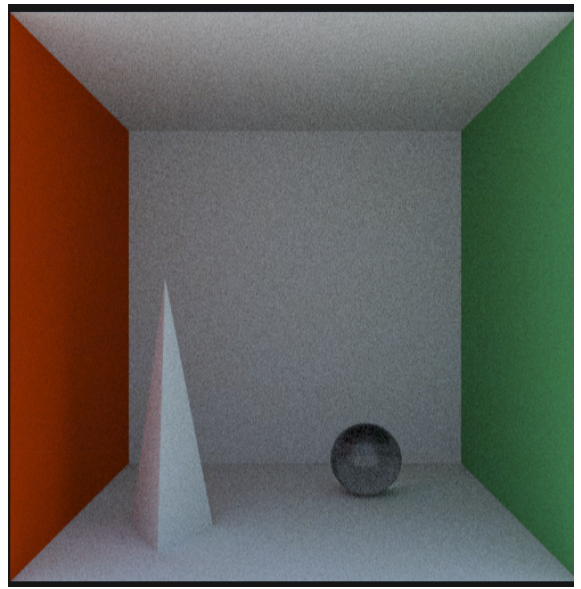
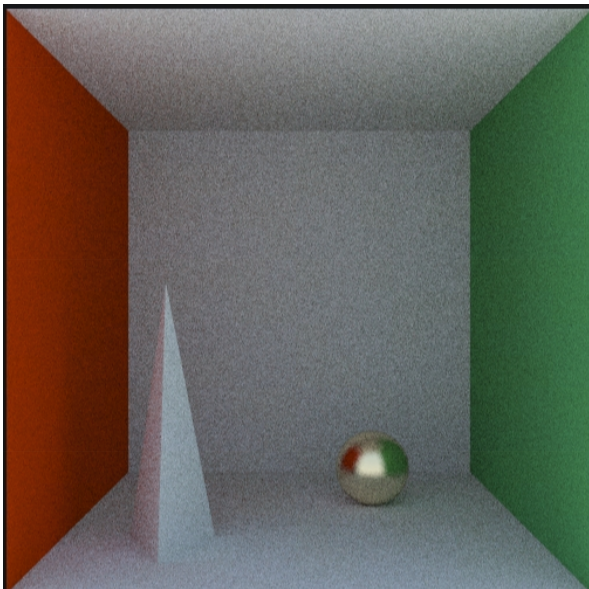
一、实验主要要求

1. 实现光线追踪算法。
2. 实现KD-tree并进行前后性能对比。

二、递归光线追踪算法的实现

主要实现了两个部件：关于不同材质的折射或者反射光线方向的计算类、光线追踪的递归算法。为了验证折射的效果，这里还添加了一个玻璃的材质。

左图为金属，右图为玻璃，这里可以看到能隐隐约约看到后面的角落的缝（取决于实现的时候折射和反射的概率）。可以看到下图中很多噪点，这里由于采样次数的不足，以及反射折射射线的范围太小，导致难以找到光源而导致某些像素是黑的。



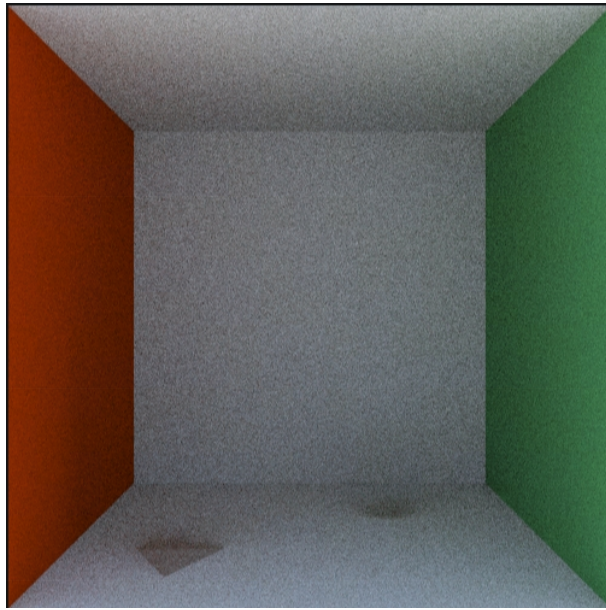
三、KD-tree的实现

KD-tree主要是建立在Oct-tree（八叉树）上的改进，不是直接一味的递推分割，而是按照原实体的位置分割节点。然后先判断叶子节点是否对光线进行碰撞再考虑节点内的。

我写完AABB盒的Oct-tree基础上继续改进为KD-tree，发现除了平面之外的原实体并不渲染，但是在光线追踪的时候应该是考虑了球和三角形的实体，因为在底平面发现有阴影的存在。

然后就是性能的问题，用KD-tree明显时间变长了，可能是我这个场景下实体太少了，直接用原本的方法更快，而不是这里的搜索子节点（因为要采样，每次都要搜索一次）。

按道理来说KD-tree应对那种三角片过多的场景，所以这个场景下未能体现优势。



四、实验的不足

1. 看图一可以看到为什么没有点光源的体现，在光线追踪的时候，即使进行了随机偏离，但是也总是难以追踪到光源，主要的光源都是来自观察方向的白光（这里没搞懂为什么）。
2. KD-tree debug了很久还是没能找到问题，只能放在后面去完成了。

五、参考文献

- https://blog.csdn.net/qq_38065509/article/details/106396844
- https://blog.csdn.net/qq_37366618/article/details/123504877
- <https://yangwc.com/2019/05/23/RayTracer-Advance/#%E4%BA%8C%E3%80%81%E4%B8%89%E8%A7%92%E7%BD%91%E6%A0%BC%E6%A8%A1%E5%9E%8B>